

END-TO-END JOB-APPLICATION MANAGEMENT SAAS

End-to-end job application management system (cloud-based MERN stack platform)

Submitted by:

AP24110010984-B.RAKESH

AP24110011041-SK.MATHIN

AP24110011068-S.TANAV REDDY

AP24110011094-G.CHARAN HARSHA

AP24110011110-D.RITHVIK DHANESH



INTRODUCTION:

The End-to-End Job-Application Management SaaS is a cloud-based and user-friendly platform built using the MERN stack (MongoDB, Express.js, React.js, Node.js) to streamline the recruitment process for both recruiters and job seekers. It provides a centralized system to manage job postings, applications, resume uploads, interview scheduling, and real-time status tracking, reducing reliance on emails and spreadsheets. The platform automates key tasks such as resume parsing, notifications, and candidate management, improving efficiency and minimizing communication gaps. It also includes secure data handling, role-based access, and analytics tools to support better decision-making, making it a scalable and effective solution for modern hiring needs.

SCENARIO BASED CASE STUDY:

Meet John Doe, a job seeker with a busy schedule who values efficiency and organization in his application process. John Doe is applying to multiple companies and needs a structured way to track job postings, applications, and interview updates.

He discovers the End-to-End Job-Application Management SaaS, a platform designed to streamline recruitment workflows. After registering and securely logging in, he creates his profile, uploads his resume, and starts applying for jobs through the platform. He can track the status of each application, receive notifications for updates, and schedule interviews seamlessly.

On the recruiter side, companies use the system to post job openings, manage candidate profiles, and track applicants through different hiring stages using an integrated pipeline. The platform also provides analytics and reporting tools to monitor hiring performance and make data-driven decisions.

With built-in automation, secure data handling, and user-friendly features, John Doe can efficiently manage his job search while recruiters can streamline their hiring process, making the platform an essential tool for modern recruitment.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the End-to-End Job-Application Management SaaS Web-Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable recruitment platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

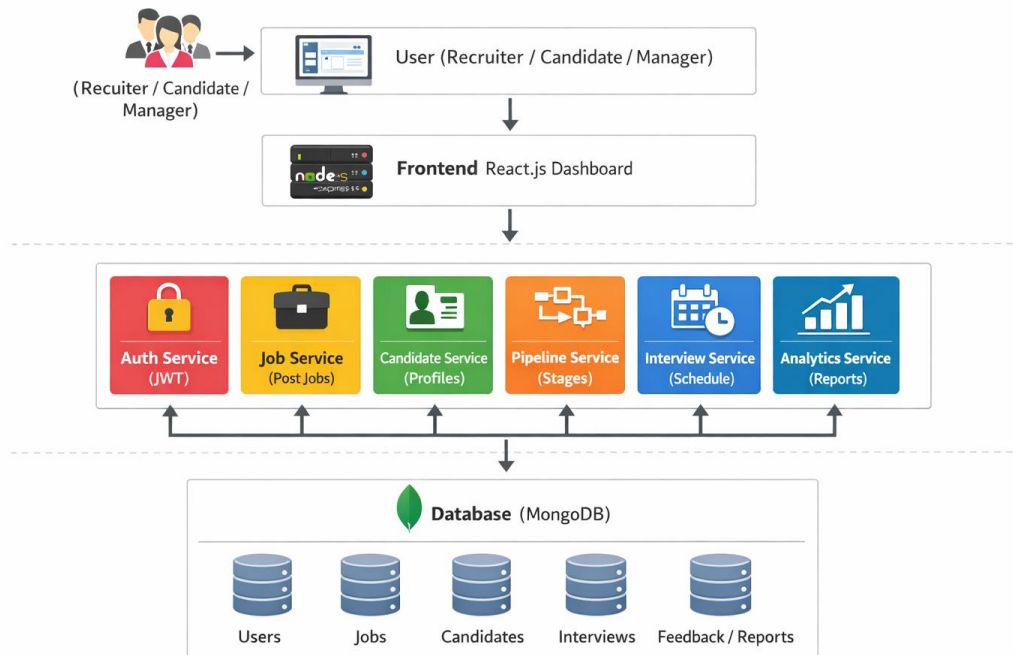
- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and managing frontend logic and powers the server-side logic and handles API routing.
- **npm (v8 or above):** A package manager required to install dependencies for React and related libraries.
- **React.js:** A JavaScript library for building dynamic and responsive user interfaces.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Express.js:** A lightweight web framework for building RESTful APIs.
- **MongoDB:** A NoSQL database used to store structured and unstructured data related to users, job postings, and applications.
- **Postman:** Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the End-to-End Job-Application Management SaaS application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures smooth performance for running development servers and handling multiple recruitment workflows.
- **RAM:** Minimum 8 GB (16 GB recommended) — Required for managing IDEs, databases, backend servers, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for project files, dependencies, and MongoDB database setup.
- **Display:** 1366x768 or higher — Recommended for better visualization of dashboards, application interfaces, and coding environment.

ARCHITECTURE:



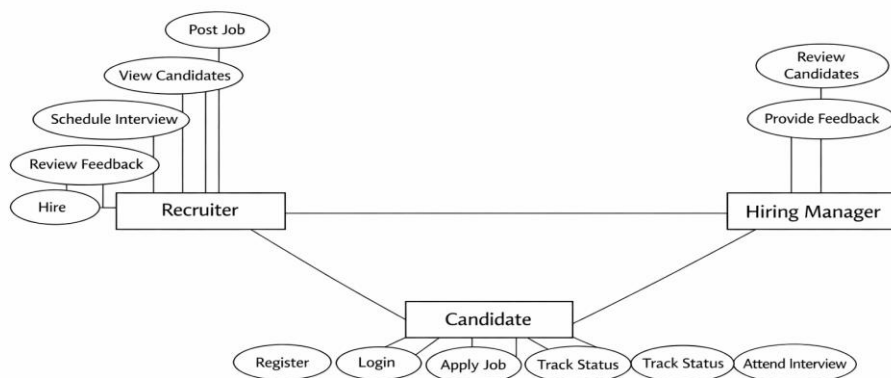
In this architecture, the flow begins with a user (recruiter, candidate, or manager) interacting with the End-to-End Job-Application Management SaaS frontend built using React.js. The user performs actions such as registering, logging in, posting jobs, applying for positions, or tracking application status through the dashboard interface.

The frontend then sends requests to the backend server, developed using Node.js and Express.js, where the application logic is processed. The backend handles various services including authentication (JWT), job management, candidate profiles, hiring pipeline stages, interview scheduling, and analytics.

The backend communicates with the MongoDB database to store and manage data such as user details, job postings, candidate information, interview schedules, and feedback reports securely. Each service interacts with the database to ensure smooth data flow and real-time updates across the system.

After processing the request, the backend sends the appropriate response back to the frontend, where the results—such as updated application status, scheduled interviews, or analytics insights—are displayed to the user. This architecture ensures a seamless, scalable, and efficient recruitment workflow for both job seekers and recruiters.

ER DIAGRAM:



The Entity-Relationship (ER) diagram for the End-to-End Job-Application Management SaaS represents the various entities and their relationships within the recruitment management system. Here is a description of the key components of the ER diagram:

1. Entities:

- **Recruiter:** Represents recruiters who manage the hiring process, with attributes such as recruiter ID, name, company, and contact information.
- **Candidate:** Represents job applicants using the platform, with attributes such as candidate ID, name, resume, skills, and contact details.
- **Job:** Represents job postings created by recruiters, with attributes such as job ID, title, description, requirements, and status.
- **Application:** Represents job applications submitted by candidates, with attributes such as application ID, date, and status.
- **Interview:** Represents interview schedules, with attributes such as interview ID, date, time, and feedback.
- **Hiring Manager:** Represents managers involved in candidate evaluation, with attributes such as manager ID, name, and contact information.

2. Relationships:

- **Recruiter–Job:** Represents the relationship where recruiters create and manage job postings.
- **Candidate–Application:** Represents candidates applying for jobs through the platform.
- **Job–Application:** Indicates which job a particular application belongs to.
- **Application–Interview:** Represents the scheduling of interviews for selected applications.
- **Hiring Manager–Interview:** Indicates that hiring managers review candidates and provide feedback during interviews.

3. Attributes:

- Each entity has its own set of attributes that describe its properties. For example, the Job entity includes attributes such as job ID, title, description, and requirements.

4. Primary Keys:

- Each entity has a primary key that uniquely identifies each record. For example, the Candidate entity is identified by candidate ID, and the Job entity by job ID.

5. Foreign Keys:

- Foreign keys are used to establish relationships between entities. For example, the Application entity contains candidate ID and job ID to link candidates with job postings.

Overall, the ER diagram for the End-to-End Job-Application Management SaaS provides a clear representation of the database structure, showing how different entities interact and how data flows through the recruitment process.

KEY FEATURES:

The End-to-End Job-Application Management SaaS offers a comprehensive set of features designed to streamline and optimize the recruitment process for both recruiters and candidates.

User Registration and Profiles:

Allows users (recruiters, candidates, and hiring managers) to create accounts, manage profiles, and store relevant personal and professional information.

Job and Application Management:

Enables recruiters to post jobs and candidates to search and apply for positions, while tracking applications based on preferences and requirements.

Resume Parsing:

Automatically extracts key information from uploaded resumes.

- Candidate profile extraction: Converts resumes into structured data such as skills, education, and experience.

Pipeline Management:

Tracks candidates throughout the hiring process.

- Stage-wise tracking: Candidates move through stages like Applied → Interview → Offer → Hired.

Interview Scheduling:

Simplifies the process of organizing interviews.

- Calendar integration: Supports scheduling through tools like Google Calendar or Outlook to avoid conflicts.

Feedback Collection:

Allows interviewers to evaluate candidates effectively.

- Centralized evaluation: Feedback is stored and used for informed hiring decisions.

Analytics & Reporting:

Provides insights into recruitment performance.

- Hiring metrics: Includes reports such as time-to-hire and source analysis for better decision-making.

Application History:

Maintains a record of all applications, interviews, and statuses for users to track progress.

Security:

Ensures safe and controlled access to the platform.

- **Role-based access:** Different roles (Recruiter, Hiring Manager, Candidate) have specific permissions and access levels.

ROLES AND RESPONSIBILITIES:

User (Candidate / Recruiter / Hiring Manager):

- **Registration and Account Management:** Users are responsible for creating accounts, providing accurate details, and managing their profiles, including updates and password security.
- **Job Search and Application:** Candidates can search for jobs based on preferences such as role, location, and skills, and apply by submitting resumes and required details. Recruiters can post jobs and manage listings.
- **Application Tracking:** Candidates track their application status, while recruiters monitor candidate progress through different hiring stages.
- **Interview Scheduling and Communication:** Users coordinate interviews and communicate through the platform for updates, queries, or scheduling changes.
- **Feedback and Reviews:** Hiring managers and recruiters provide feedback on candidates, while candidates may share their experience of the hiring process.

Admin:

- **System Management:** The admin manages overall system performance, ensures data security, and maintains platform configurations.
- **User Management:** Responsible for handling user accounts, including registration, verification, approvals, and suspensions if required.
- **Job and Recruitment Management:** Oversees job postings, monitors recruitment activities, and ensures proper functioning of the hiring pipeline.

User Flow:**Candidate Flow**

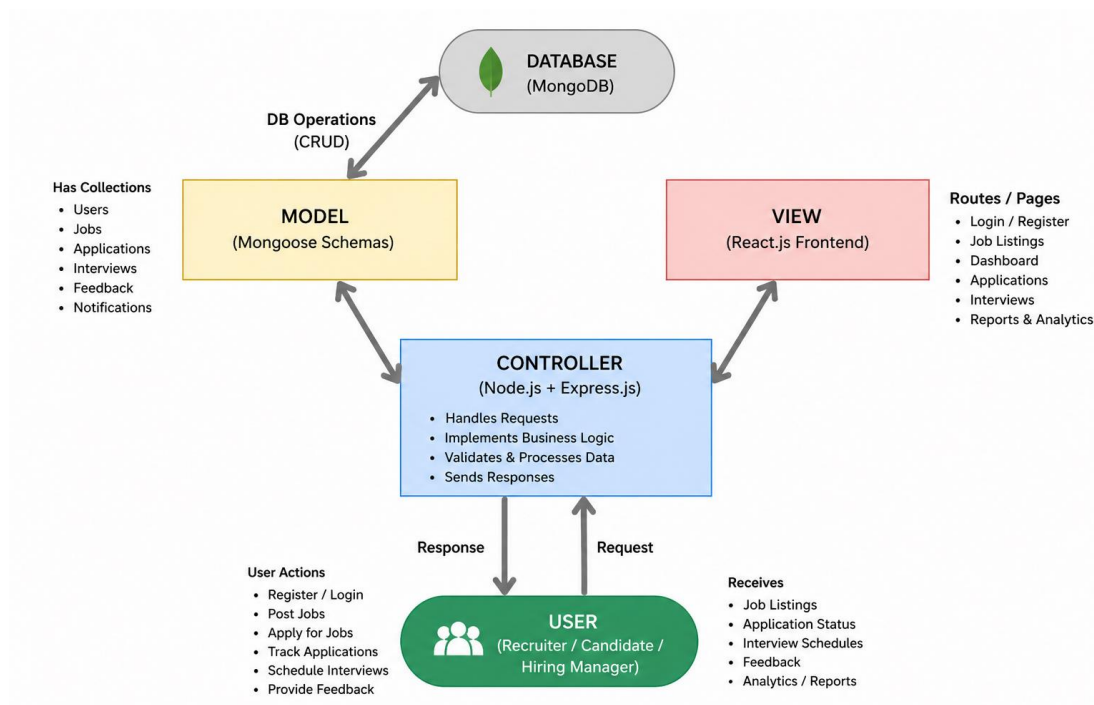
- **Registration & Account:** Create and manage profile and upload resume.
- **Job Search & Apply:** Browse jobs and submit applications.
- **Track Status:** Monitor application progress in real time.

- **Interview:** Attend scheduled interviews.
- **Feedback:** View feedback or updates from recruiters.

Recruiter/Admin Flow

- **System Management:** Monitor platform performance and maintain security.
- **User Management:** Verify and manage user accounts.
- **Job Management:** Post jobs and manage applications.
- **Candidate Evaluation:** Review candidates, schedule interviews, and provide feedback.

MVC PATTERN:



The End-to-End Job-Application Management SaaS follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates the application into three interconnected layers. This separation ensures

modularity, improves maintainability, and enhances scalability, allowing the system to efficiently handle recruitment workflows, user interactions, and data management.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic in the End-to-End Job-Application Management SaaS. It defines schemas for entities such as users, jobs, applications, interviews, and feedback, and manages database operations using Mongoose with MongoDB.

Controller Layer

The Controller layer acts as an intermediary between the routes (view layer) and the model. It processes incoming requests, performs validation and business logic (such as applying for jobs or scheduling interviews), interacts with the models, and returns appropriate responses to the client.

View Layer (Routing Layer)

In this project, the View layer is implemented as the routing layer in the backend REST API. It defines endpoints for handling HTTP requests (GET, POST, PUT, DELETE) and maps them to the corresponding controller functions. The frontend (React.js) interacts with these routes to display data to users.

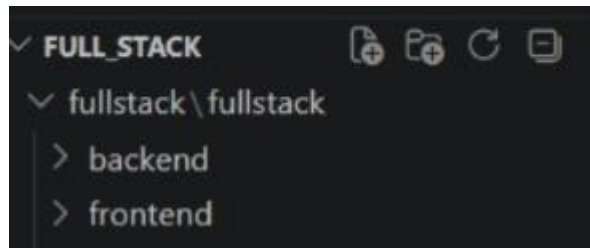
Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a distinct responsibility, improving code organization and maintainability.
- **Scalability:** New features like additional modules or services can be easily integrated.
- **Reusability:** Controllers and models can be reused across different parts of the system.
- **Testing:** Each layer can be tested independently for better reliability.
- **Collaboration-Friendly:** Developers can work on frontend, backend logic, and database independently without conflicts.

PROJECT SETUP AND CONFIGURATION:

Creating Project Folder (Updated)

1. Create a new folder with your **<project name>**.
2. Inside that folder, create another folder named **fullstack**.
3. Inside the **fullstack** folder, create two new folders.
4. Name one folder as **frontend**.
5. Name the other folder as **backend**.
6. Now open the **<project name>** folder in VS Code.



Frontend setup (installing react app)

- Open the **frontend** folder in the terminal of VS Code.
 - Run the following command:
■ `npm create vite@latest . -- --template react`
 - Select React framework from the given options.
 - Select a framework:
■ | React
 - Select JavaScript variant from the given options.
 - Select a variant:
■ | JavaScript
 - To install all the packages run the following command.
■ `npm install`
 - To start the React server type the following command.
■ `npm run dev`

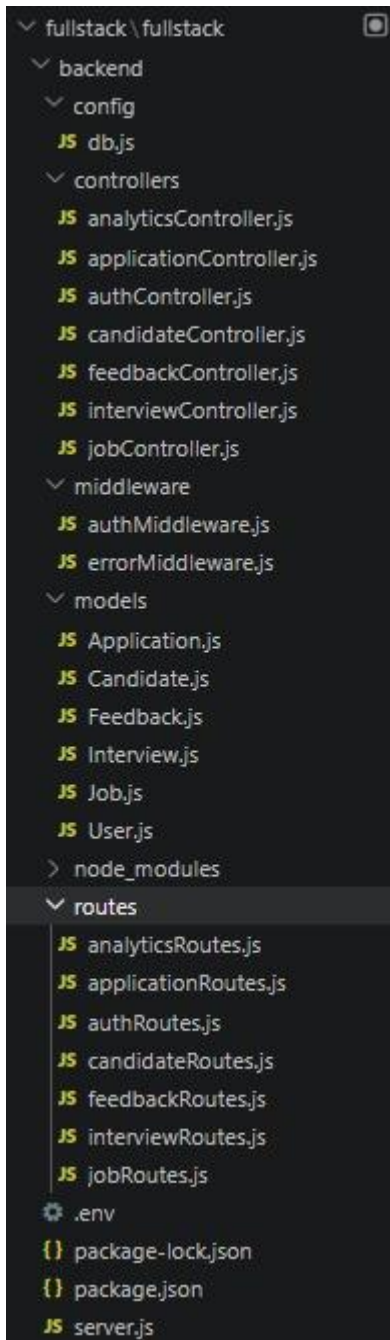
Frontend files

- Languages used:
 - HTML5, CSS3, JavaScript (ES6+)
- Core files:
 - index.html → main entry point
 - style.css → design, UI, animations
 - main.jsx / react.js → core React logic
- Functionality:
 - State management (user roles, login status)
 - Component rendering (Candidate, Recruiter, Manager dashboards)
 - AI assistant integration
 - Modern UI (fonts, colors, micro-interactions)

Backend setup (npm init)

- Open **backend** folder in terminal of VS Code.
 - npm init -y
- Install required packages:
 - npm install express mongoose cors dotenv bcrypt jsonwebtoken
- Create files:
 - server.js
- Create folders:
 - models
 - controllers
 - routes

Backend Structure:



Controllers

- analyticsController.js → Handles analytics data (dashboard stats, reports, insights).
- applicationController.js → Manages job applications (apply, update status, track applications).

- `authController.js` → Handles authentication (register, login, role-based access).
- `candidateController.js` → Handles candidate-related operations (profile, applications, activity).
- `feedbackController.js` → Manages feedback system (submit, view feedback).
- `interviewController.js` → Handles interview scheduling and status updates.
- `jobController.js` → Manages job postings (create, update, delete, fetch jobs).

db / config

- `db.js` → Contains database connection setup (MongoDB connection using Mongoose).

middleware

- `authMiddleware.js` → Middleware for authentication (JWT validation, protected routes).
- `errorMiddleware.js` → Handles global error handling and response formatting.

models

- `Application.js` → Schema/model for job applications.
- `Candidate.js` → Schema/model for candidate profiles and details.
- `Feedback.js` → Schema/model for feedback data.
- `Interview.js` → Schema/model for interview scheduling and results.
- `Job.js` → Schema/model for job postings.
- `User.js` → Schema/model for user accounts (with roles like candidate, recruiter, manager).

routes

- `analyticsRoutes.js` → Defines API routes for analytics and dashboard data.
- `applicationRoutes.js` → Defines API routes for job applications.
- `authRoutes.js` → Defines API routes for authentication (login, register).
- `candidateRoutes.js` → Defines API routes for candidate operations.
- `feedbackRoutes.js` → Defines API routes for feedback system.

- `interviewRoutes.js` → Defines API routes for interview management.
- `jobRoutes.js` → Defines API routes for job-related actions.

root files

- `server.js` → Entry point of backend, initializes Express server, middleware, and routes.
- `.env` → Stores environment variables (DB URL, JWT secret, etc.).
- `package.json` → Project configuration and dependencies.

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- Install Mongoose.

In your Node.js project directory, run: `npm install mongoose`

- Create database connection
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

Create Schemas:

○ Now, make sure the database is connected before performing any backend operations.

○ In your case (as shown in the image), MongoDB is connected using:

■ `mongodb://localhost:27017/`

○ Create a database connection file inside:

■ `backend/config/db.js`

New Connection

Manage your connection settings

URI ⓘ

Edit Connection String ☒

mongodb://localhost:27017/

Name

JAMS

Color

No Color

☐ Favorite this connection

Favoriting a connection will pin it to the top of your list of connections

[Advanced Connection Options](#)

Cancel

Save

Connect

Save & Connect

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

How do I format my connection string?

[See example](#)

1) Users.js → Schema/model for user accounts.

```
import bcrypt from "bcryptjs";
import mongoose from "mongoose";

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, lowercase: true, trim: true },
    password: { type: String, required: true, minlength: 6 },
    role: {
      type: String,
      enum: ["Recruiter", "Hiring Manager", "Candidate", "Admin"],
      default: "Candidate",
    },
  },
  { timestamps: true }
);

// Same email can register with different roles, but not the same role twice
userSchema.index({ email: 1, role: 1 }, { unique: true });

userSchema.pre("save", async function hashPassword(next) {
  if (!this.isModified("password")) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.matchPassword = function matchPassword(password) {
  return bcrypt.compare(password, this.password);
};

export default mongoose.model("User", userSchema);
```


2) analyticsController.js → Handles analytics overview (counts jobs, candidates, interviews, and pipeline stages).

```
# applicationControllers.js
fullstack > fullstack > backend > controllers > # applicationControllers.js > ...

1 import Application from "../models/Application.js";
2
3 export async function createApplication(req, res, next) {
4   try {
5     const payload = {
6       ...req.body,
7       candidateUser: req.user.role === "Candidate" ? req.user._id : req.body.candidateUser,
8       lastActivityAt: new Date(),
9     };
10
11     const application = await Application.create(payload);
12     res.status(201).json(application);
13   } catch (error) {
14     if (error.code === 11000) {
15       return res.status(400).json({ message: "Application already exists for this candidate and job" });
16     }
17     next(error);
18   }
19 }
20
21 export async function getApplications(req, res, next) {
22   try {
23     const query = {};
24
25     if (req.user.role === "Candidate") {
26       query.candidateUser = req.user._id;
27     }
28
29     if (req.query.status) {
30       query.status = req.query.status;
31     }
32
33     if (req.query.job) {
34       query.job = req.query.job;
35     }
36
37     const applications = await Application.find(query)
38       .populate("candidateUser", "name email role")
39       .populate("candidateProfile", "name email parsedData score")
40       .populate("job", "title department location status")
41       .sort({ updatedAt: -1 });
42
43     res.json(applications);
44   } catch (error) {
45     next(error);
46   }
47 }
48
49 export async function getApplication(req, res, next) {
50   try {
51     const application = await Application.findById(req.params.id)
52       .populate("candidateUser", "name email role")
53       .populate("candidateProfile", "name email parsedData score")
54       .populate("job", "title department location status");
55
56     if (!application) {
57       return res.status(404).json({ message: "Application not found" });
58     }
59
60     if (
61       req.user.role === "Candidate" &&
62       String(application.candidateUser?._id || application.candidateUser) !== String(req.user._id)
63     ) {
64       return res.status(403).json({ message: "Forbidden for this role" });
65     }
66
67     res.json(application);
68   } catch (error) {
69     next(error);
70   }
71 }
72
73 export async function updateApplication(req, res, next) {
74   try {
75     const application = await Application.findById(req.params.id);
76     if (!application) {
77       return res.status(404).json({ message: "Application not found" });
78     }
79
80     if (
81       req.user.role === "Candidate" &&
82       String(application.candidateUser?._id || application.candidateUser) !== String(req.user._id)
83     ) {
84       return res.status(403).json({ message: "Forbidden for this role" });
85     }
86
87     Object.assign(application, req.body, { lastActivityAt: new Date() });
88     await application.save();
89
90     res.json(application);
91   } catch (error) {
92     next(error);
93   }
94 }
```

3) Application.js → Schema/model for job applications (tracks candidate, job, status, resume, and application stages).

```
JS Application.js X
fullstack > fullstack > backend > models > JS Application.js > ...
1 import mongoose from "mongoose";
2
3 const applicationSchema = new mongoose.Schema(
4   {
5     candidateUser: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
6     candidateProfile: { type: mongoose.Schema.Types.ObjectId, ref: "Candidate" },
7     job: { type: mongoose.Schema.Types.ObjectId, ref: "Job", required: true },
8     status: {
9       type: String,
10      enum: ["Applied", "Screening", "Interview", "Offer", "Hired", "Rejected", "Withdrawn"],
11      default: "Applied",
12    },
13    source: { type: String, default: "Careers page" },
14    resumeUrl: { type: String },
15    coverLetter: { type: String },
16    notes: { type: [String], default: [] },
17    recruiterTags: { type: [String], default: [] },
18    lastActivityAt: { type: Date, default: Date.now },
19  },
20  { timestamps: true }
21 );
22
23 applicationSchema.index({ candidateUser: 1, job: 1 }, { unique: true });
24
25 export default mongoose.model("Application", applicationSchema);
26
```

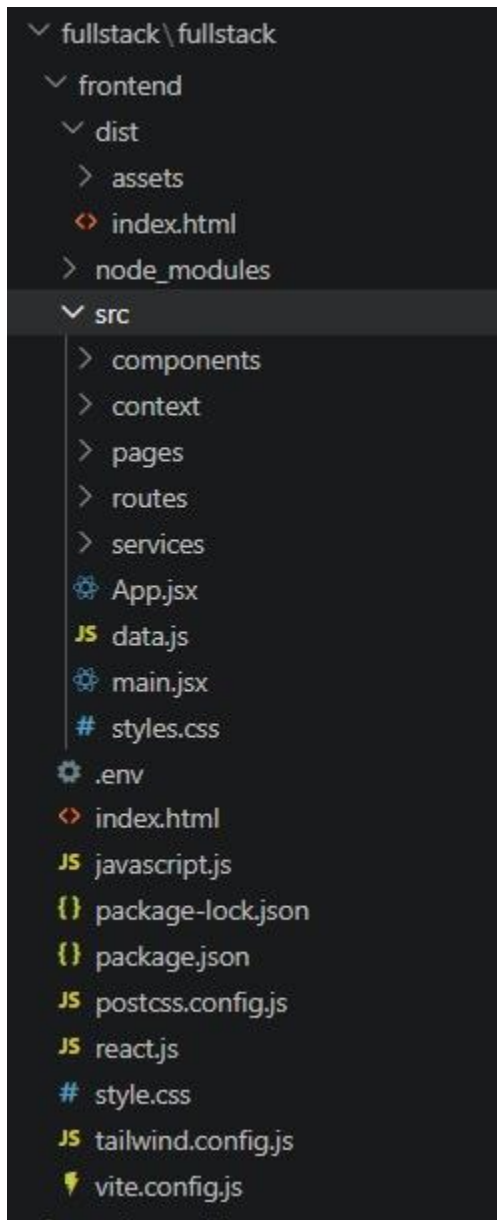
4) Job.js → Schema/model for job postings (includes title, description, status, stages, priority, and hiring details).

```
JS Job.js X
fullstack > fullstack > backend > models > JS Job.js > ...
1 import mongoose from "mongoose";
2
3 const jobSchema = new mongoose.Schema(
4   {
5     title: { type: String, required: true, trim: true },
6     description: { type: String, required: true },
7     department: { type: String, required: true },
8     location: { type: String, default: "Remote" },
9     status: { type: String, enum: ["Draft", "Published", "Screening", "Interviewing", "Closed"], default: "Draft" },
10    approvalStatus: { type: String, enum: ["Pending", "Approved", "Rejected"], default: "Pending" },
11    priority: { type: String, enum: ["Low", "Medium", "High"], default: "Medium" },
12    stages: {
13      type: [String],
14      default: ["Applied", "Screening", "Interview", "Offer", "Hired"],
15    },
16    openings: { type: Number, min: 1, default: 1 },
17    employmentType: { type: String, default: "Full-time" },
18    skills: { type: [String], default: [] },
19    current: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
20  }, { timestamps: true }
21 );
22
23 export default mongoose.model("Job", jobSchema);
24
25
```

5) authRoutes.js → Defines API routes for authentication (register with validation, login, and protected user profile).

```
JS authRoutes.js X
fullstack > fullstack > backend > routes > JS authRoutes.js > ...
1  import express from "express";
2  import { body } from "express-validator";
3  import { login, me, register } from "../controllers/authController.js";
4  import { protect } from "../middleware/authMiddleware.js";
5
6  const router = express.Router();
7
8  router.post(
9    "/register",
10   [
11     body("name").trim().notEmpty().withMessage("Name is required"),
12     body("email").isEmail().withMessage("Valid email is required"),
13     body("password").isLength({ min: 6 }).withMessage("Password must be at least 6 characters"),
14     body("role")
15       .isIn(["Recruiter", "Hiring Manager", "Candidate"])
16       .withMessage("Valid role is required"),
17   ],
18   register
19 );
20
21 router.post("/login", login);
22 router.get("/me", protect, me);
23
24 export default router;
25
```

FRONTEND STRUCTURE:



components

- Footer.jsx → Reusable footer component used across pages.
- (ui/) → Contains shared UI components (buttons, modals, reusable elements).

context

- context/ → Manages global state (user data, auth state, roles, app-wide data).

pages

- pages/ → Contains main application pages (dashboard views, role-based screens like Candidate, Recruiter, Manager).

routes

- routes/ → Handles frontend routing (navigation between pages using React Router).

services

- services/ → Handles API calls to backend (auth, jobs, applications, etc.).

core files

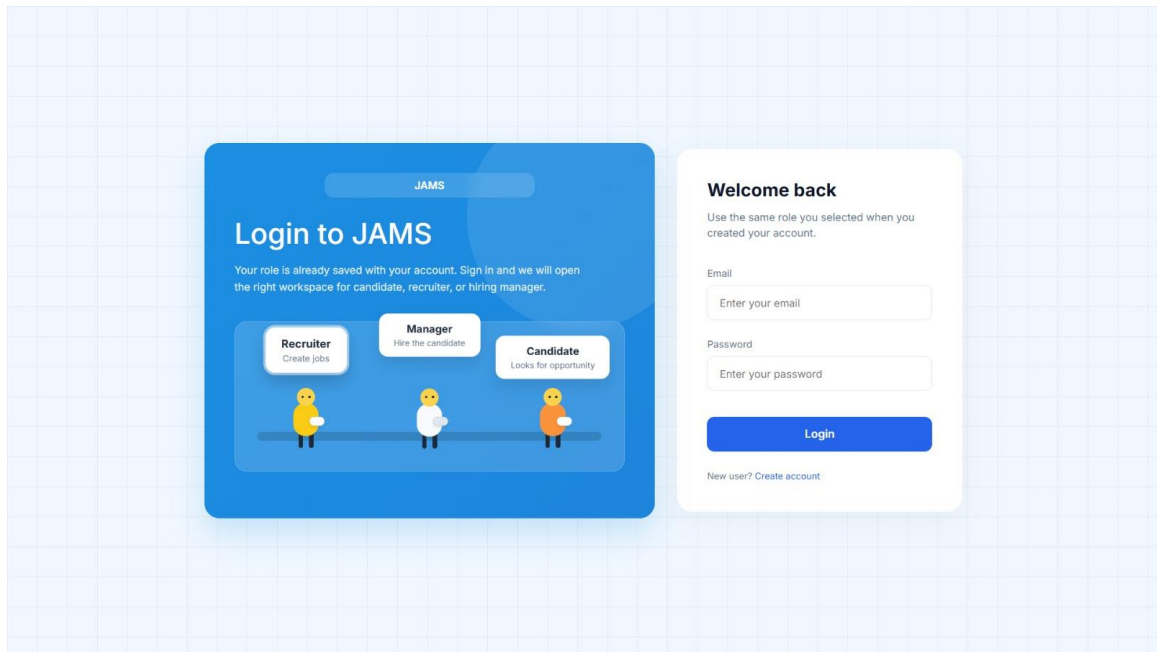
- App.jsx → Main app component (defines layout, routes, and structure).
- main.jsx → Entry point of React app (renders App component).
- styles.css → Global styling (theme, layout, UI design).
- data.js → Static/mock data used in the application.

root files

- index.html → Main HTML file (root container for React app).
- javascript.js → Additional JS logic (if used outside React).
- react.js → Core React-related logic/helpers.
- style.css → Additional styling file.
- vite.config.js → Vite configuration.
- tailwind.config.js → Tailwind CSS configuration.
- postcss.config.js → PostCSS configuration.
- package.json → Project dependencies and scripts.
- .env → Environment variables (API URLs, keys).

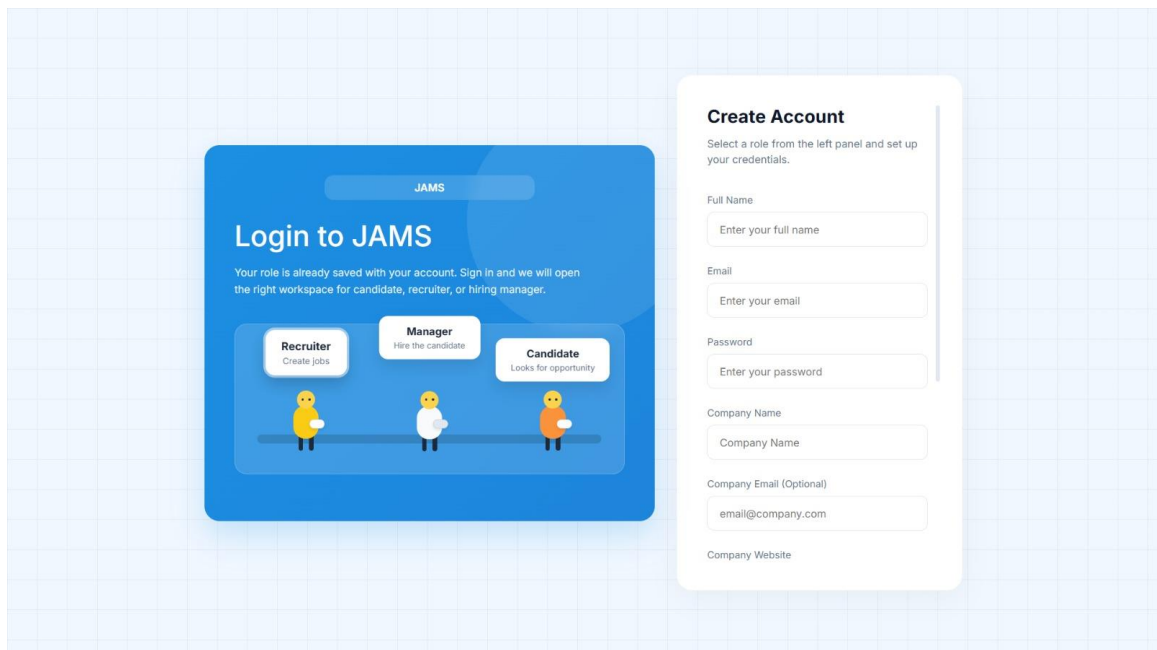
OUTPUT SCREENSHOTS:

Login.jsx



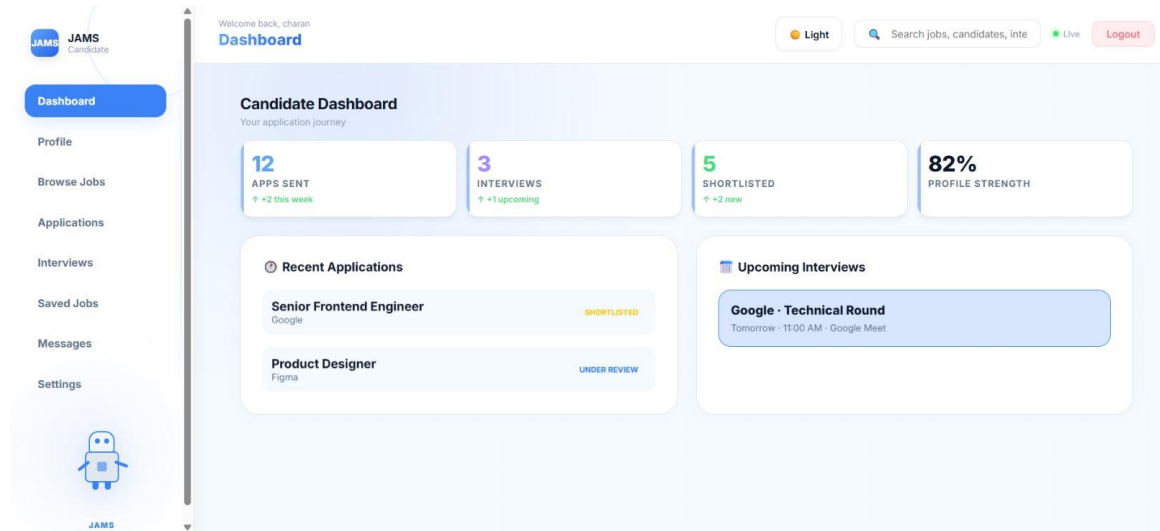
The screenshot displays the 'Login to JAMS' interface. On the left, a blue panel features the JAMS logo and a title 'Login to JAMS'. Below the title, a message states: 'Your role is already saved with your account. Sign in and we will open the right workspace for candidate, recruiter, or hiring manager.' Three role selection buttons are shown: 'Recruiter' (Create jobs), 'Manager' (Hire the candidate), and 'Candidate' (Looks for opportunity), each accompanied by a small icon. On the right, a white panel titled 'Welcome back' contains a message: 'Use the same role you selected when you created your account.' It includes input fields for 'Email' and 'Password', a blue 'Login' button, and a link for 'New user? Create account'.

Register.jsx

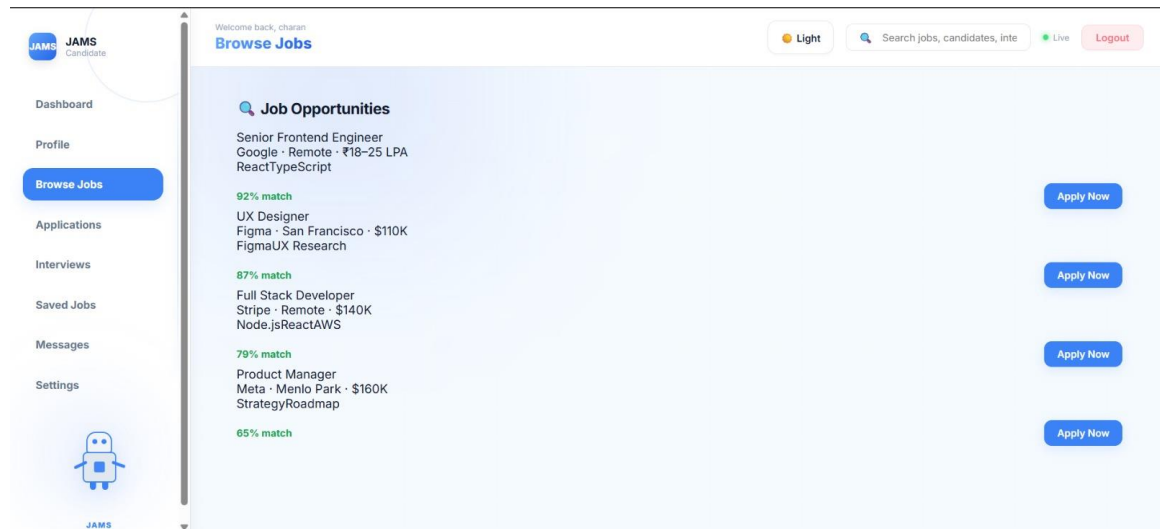


The screenshot displays the 'Create Account' interface. On the left, a blue panel features the JAMS logo and a title 'Login to JAMS'. Below the title, a message states: 'Your role is already saved with your account. Sign in and we will open the right workspace for candidate, recruiter, or hiring manager.' Three role selection buttons are shown: 'Recruiter' (Create jobs), 'Manager' (Hire the candidate), and 'Candidate' (Looks for opportunity), each accompanied by a small icon. On the right, a white panel titled 'Create Account' contains a message: 'Select a role from the left panel and set up your credentials.' It includes input fields for 'Full Name', 'Email', 'Password', 'Company Name', 'Company Email (Optional)' (with a placeholder 'email@company.com'), and 'Company Website'.

Candidate homepage:



Browse Job(Candidate):



Applications(Candidate):

Welcome back, charan

Applications

Light Search jobs, candidates, inte Live Logout

My Applications

COMPANY	ROLE	STATUS	APPLIED	ACTION
Google	Senior Frontend Engineer	SHORTLISTED	Apr 20	Withdraw
Figma	Product Designer	UNDER REVIEW	Apr 22	Withdraw
Stripe	Full Stack Developer	APPLIED	Apr 25	Withdraw
Meta	Product Manager	REJECTED	Apr 18	Withdraw

JAMS

Interviews(Candidate):

Welcome back, charan

Interviews

Light Search jobs, candidates, inte Live Logout

Interviews

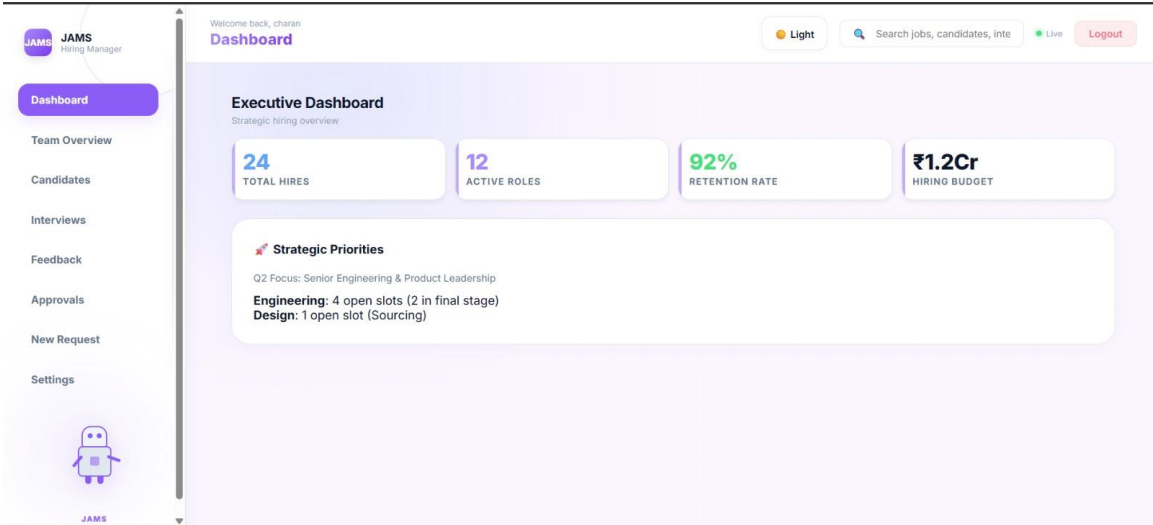
Upcoming

Google · Technical Round
Tomorrow · 11:00 AM · Google Meet

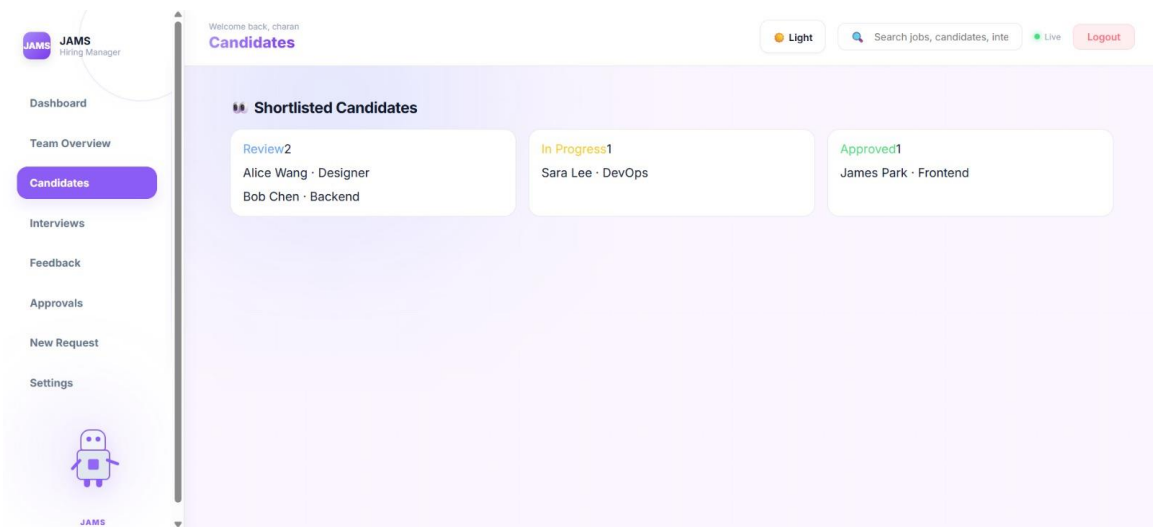
Figma · Design Challenge
May 5 · 3:00 PM · Zoom

JAMS

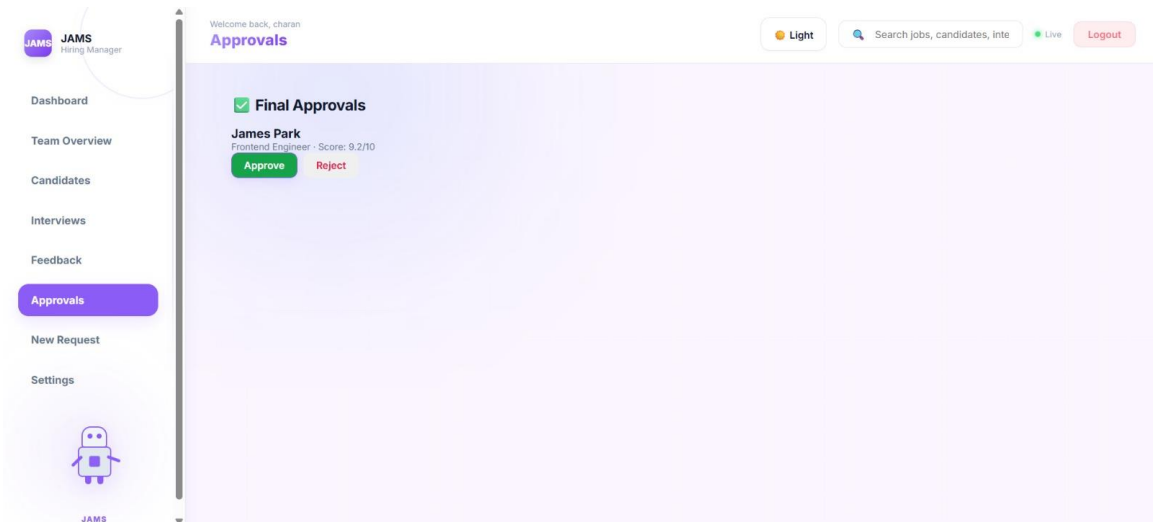
Hiring Manager homepage:



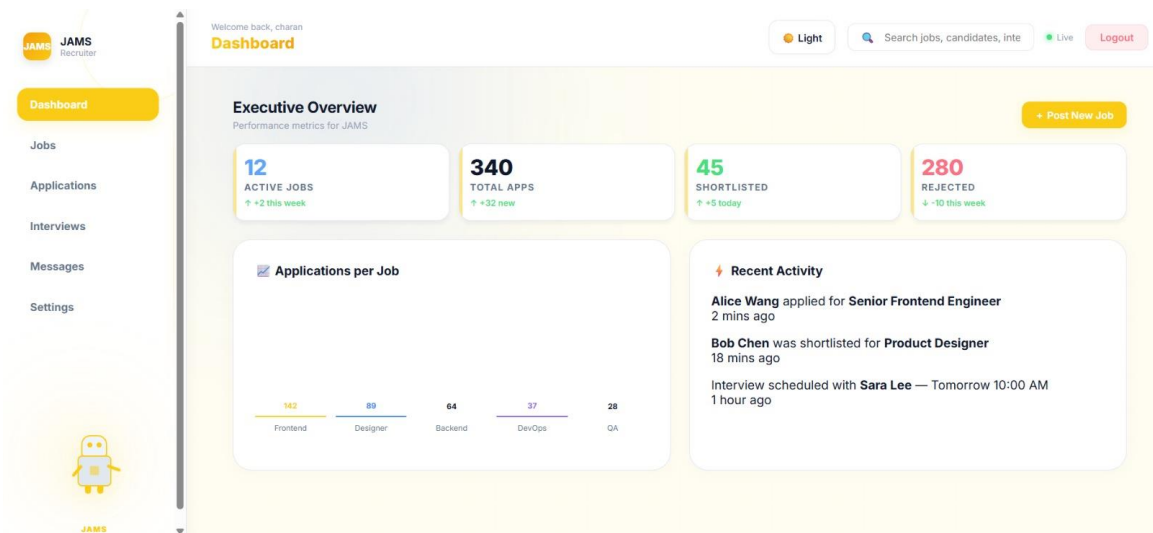
Candidate(Hiring Manager):



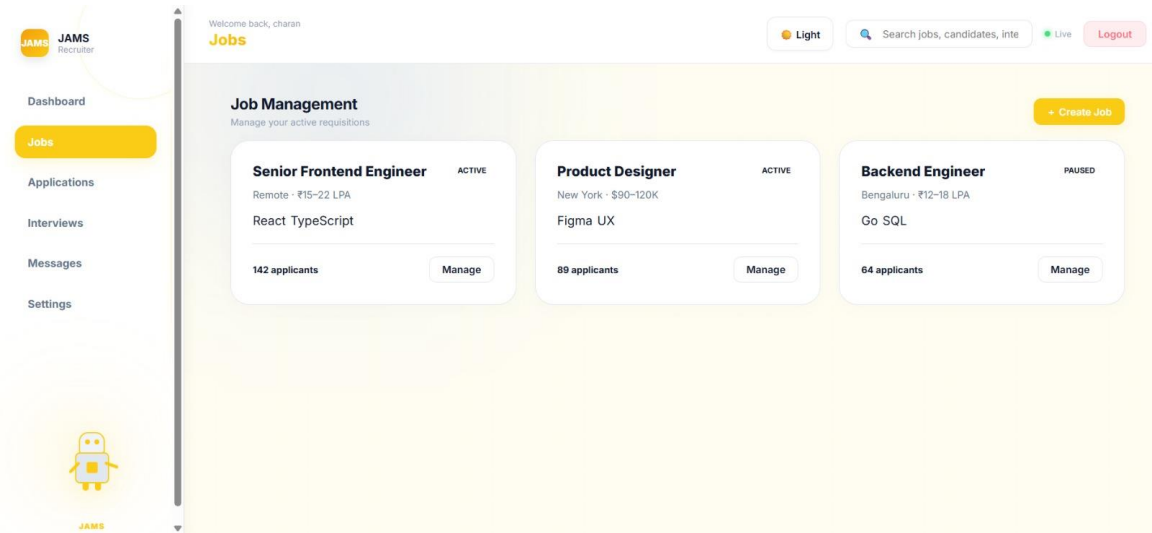
Approvals:



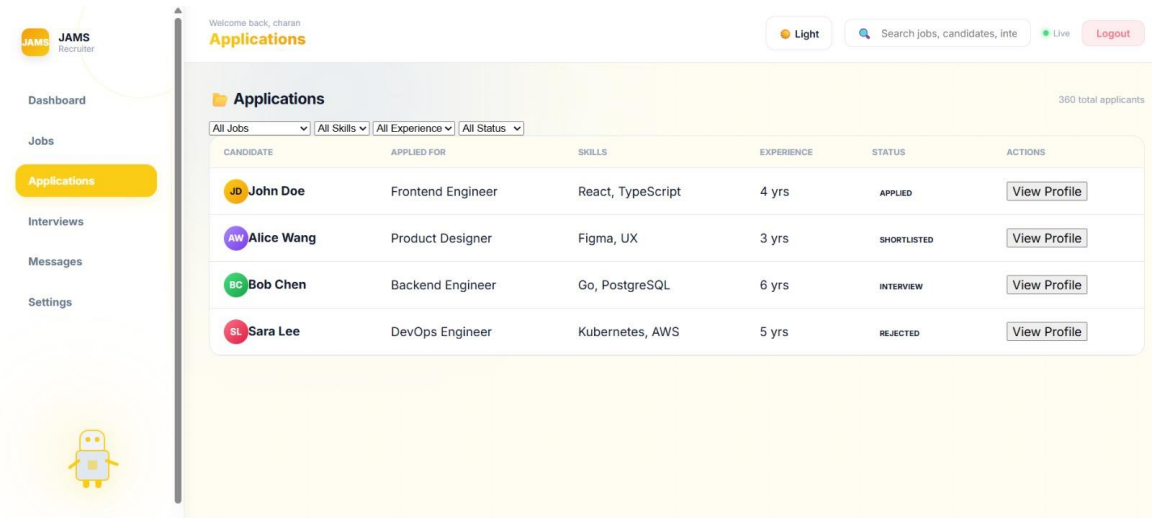
Recruiter Homepage:



Recruiter(Jobs):



Recruiter(Applications):



Recruiter(Messages):

